FU EVENT MANAGEMENT

PROJECT DOCUMENTATION AND SOFTWARE REQUIREMENTS SPECIFICATION

Software Project (SWP) | FPT University | Version 1.0 | October 2026

DOCUMENT CONTROL

Item	Description
Project Name	FU Event Management
Document Type	Software Project Documentation and SRS
Frontend	React, TypeScript, Vite, Tailwind CSS, Redux Toolkit
Backend (original project)	Java, Spring Boot, RESTful APIs, role-based access control
Database	MySQL
Frontend Deployment	Vercel, as documented in the project description
Backend Deployment	No public backend deployment documented
Document Status	Reconstructed report; implementation and tests require verification

DOCUMENT STATUS AND DISCLAIMER

This report reconstructs the technical documentation for FU Event Management from the supplied project documentation, requirements, and database-related materials. It follows the broad report structure of the provided reference report while replacing project-specific content with FU Event Management information.

The report documents intended scope, requirements, architecture, database design, testing procedures, and local setup. It does not claim that every feature is functional or that the database script has been validated against the original production system. Functional requirements FR-01 to FR-19 are aligned with the revised User Requirements Document; payment behavior, exact authorization rules, and implementation-dependent details remain subject to source-code verification.

TABLE OF CONTENTS

I. PROJECT INTRODUCTION 4
1. Project Overview 4
2. Project Background and Problem Statement 4
3. Project Objectives 4
4. Project Scope 4
5. Stakeholders and User Classes 5
6. Project Deliverables and Success Criteria 5
II. PROJECT MANAGEMENT PLAN 5
III. SOFTWARE REQUIREMENTS SPECIFICATION 7
1. Product Overview 7
2. User Requirements 7
3. Functional Requirements 9
3.2 Non-Functional Requirements 10
IV. SOFTWARE DESIGN DESCRIPTION 11
1. System Design 11
2. Database Design 13
2.3 Overall Entity Relationship Diagram 14
2.4 Main User Workflow Diagram 15
2.5 Principal Database Relationships 16
V. SOFTWARE TESTING DOCUMENTATION 17
2. Test Plan 18
VI. RELEASE PACKAGE AND USER GUIDES 19
2. Installation Guides 19
3. User Manual 20
VII. PROJECT EVALUATION, LIMITATIONS, AND FUTURE DEVELOPMENT 22
REFERENCES 23
APPENDIX A. DOCUMENTATION DELIVERABLE CHECKLIST 23
APPENDIX B. RECOMMENDED DOCUMENTATION STRUCTURE 23

I. PROJECT INTRODUCTION

1. Project Overview

FU Event Management is a web-based event management application developed as a team project for the Software Project (SWP) course at FPT University. The intended platform centralizes event information and supports participant registration, ticket management, attendance tracking, sponsorship activities, feedback collection, and surveys.

The frontend is built with React and TypeScript using Vite, Tailwind CSS, and Redux Toolkit. The original backend uses Java and Spring Boot to expose RESTful APIs, while MySQL provides relational data storage. The frontend deployment platform is documented as Vercel; a public backend deployment has not been established in the available materials.

2. Project Background and Problem Statement

University events involve coordination among event operators, visitors, checking staff, sponsors, and administrators. When event information, participant lists, tickets, attendance, and feedback are maintained in separate tools, data may become fragmented, duplicated, or inconsistent. FU Event Management was designed to provide a centralized application and relational data model for these activities.

3. Project Objectives

- Provide a web-based platform for managing university event information.
- Support event discovery and participant-related workflows.
- Represent tickets and attendance records in a structured database.
- Organize sponsor profiles, sponsor-event associations, and sponsor programs.
- Provide data structures for feedback forms, questions, and visitor answers.
- Represent surveys, survey questions, and survey answer records.
- Support account-role associations and authorization where implemented.
- Document a reproducible local setup and a maintainable database baseline.

4. Project Scope

4.1 In-Scope Functionality

- Event information, lifecycle state, images, schedules, sale windows, dates, and prices.
- Visitor profiles, ticket records, ticket status, and relevant timestamps.
- Attendance records and checking-staff associations with events.
- Sponsor profiles, sponsor-event relationships, and sponsor programs.
- Event fundraising information and event-level financial summary records.
- Feedback forms, feedback questions, and visitor answers.
- Surveys, survey questions, and answer-related records.
- Database documentation, local configuration, and technical verification.

4.2 Out-of-Scope or Unverified Functionality

- Production payment gateway integration and financial reconciliation.
- Complete donation transaction processing.
- Exact email notification and account recovery behavior.
- Production monitoring and observability.
- Public backend deployment.

- Exact permissions for every role and all user-interface workflows.
- Production readiness of every business process.

5. Stakeholders and User Classes

Stakeholder	Responsibilities / Interests
Visitor	Browse events, use supported registration/ticket functions, and submit feedback or survey responses.
Event Operator	Maintain event information and schedules.
Checking Staff	Check tickets and record attendance for assigned events.
Sponsor	Maintain sponsor information and sponsor programs associated with events.
Administrator	Manage accounts, roles, and system-level access where supported.
Developers	Maintain frontend, backend, database mappings, and documentation.
Project Reviewers	Evaluate requirements, architecture, database design, and implementation evidence.

6. Project Deliverables and Success Criteria

- Frontend and backend source repositories.
- User Requirements Document and project README.
- Overall logical ERD and main user workflow diagram.
- Use Case Diagram.
- MySQL database creation script and schema documentation.
- Installation instructions and technical report.
- Test records collected from actual execution.

A successful local restoration should allow frontend dependencies to be installed, the database schema to be created on a clean local MySQL instance, the backend to start with valid configuration, and the frontend to communicate with the API. Core workflows should be exercised before they are described as operational. These are target criteria, not verified outcomes.

II. PROJECT MANAGEMENT PLAN

1. Management Approach

This report supports restoration and technical review of an existing university project. Management activities therefore focus on source inspection, requirements recovery, database verification, diagram maintenance, local reproducibility, and documentation review. Historical schedules, assignments, effort estimates, and sprint outcomes are not asserted because supporting records were not provided.

2. Work Breakdown Structure

Work Package	Description	Expected Deliverable
WP-01: Source Inspection	Review frontend and backend repositories.	Source inventory and implementation notes.
WP-02: Requirements Recovery	Identify actors, intended features, and workflows.	User Requirements Document.
WP-03: Database Analysis	Review SQL definitions, entity mappings, and relationships.	Schema documentation and database script.
WP-04: Diagram Preparation	Represent entities and use cases visually.	ERDs and Use Case Diagram.
WP-05: Environment Setup	Configure MySQL, Java, and Node.js environments.	Local setup instructions.
WP-06: Integration Verification	Test frontend/backend/database integration.	Test records and issue list.

Work Package	Description	Expected Deliverable
WP-07: Documentation Review	Compare the report with source code and SQL.	Updated project documentation.

3. Risk Management

Risk	Impact	Mitigation
SQL differs from JPA mappings	Startup or persistence errors.	Compare SQL definitions with entity annotations.
Destructive SQL statements	Existing local data may be lost.	Review DROP/CREATE statements and back up data.
Exposed credentials	Unauthorized access or account compromise.	Rotate exposed credentials and use local environment variables.
Inferred requirements are inaccurate	Documentation misrepresents behavior.	Validate against controllers, services, and frontend screens.
Demo data violates constraints	SQL execution may fail.	Insert parent records before dependent records; test on a clean schema.
Diagrams become outdated	Reviewers may be misled.	Regenerate diagrams after schema changes.
Incomplete local configuration	Application may not start.	Document required versions and environment variables.

4. Quality Management

- Review consistency of terminology, requirements, diagrams, and schema documentation.
- Check primary keys, foreign keys, nullability, defaults, and status values.
- Validate the SQL script against the intended MySQL version.
- Review configuration files for exposed secrets.
- Test backend startup and frontend dependency installation.
- Execute smoke tests for critical workflows and record unresolved defects.

5. Configuration Management and Communication

- Maintain frontend, backend, SQL scripts, diagrams, and documentation under version control.
- Reflect database changes in schema documentation and ERDs.
- Reflect API behavior changes in requirements and use case specifications.
- Keep passwords, signing keys, and SMTP credentials out of public repositories.
- Use consistent names for actors, tables, roles, and workflows.

III. SOFTWARE REQUIREMENTS SPECIFICATION

1. Product Overview

FU Event Management is described as a web application with a presentation layer, an application layer, and a relational data layer. The React frontend presents screens and forms; the Spring Boot backend processes HTTP requests and business rules; MySQL stores application records. Exact endpoint paths and request/response contracts must be obtained from the source code.

2. User Requirements

2.1 Actors

Actor	Description
Guest	Views events and news; registers an account.

Actor	Description
Registered User	Logs in and logs out.
Visitor	Browses events and news, views event and fundraising information, registers for events, manages tickets, and submits feedback or survey responses.
Event Operator	Manages event details and schedules.
Checking Staff	Checks tickets and records attendance for assigned events.
Sponsor	Manages sponsor profile/program information and views permitted sponsorship associations.
Administrator	Manages accounts and roles; system-configuration permissions require source verification.

2.2 Representative Use Case Descriptions

The following seven use cases are representative high-level narratives, not a complete enumeration of every use case in Figure 1. Detailed steps, permissions, and failure paths must be verified against the actual UI and API behavior.

UC-01: Browse Events

Actor: Visitor. The visitor opens the event list; the frontend requests event information; the backend returns available records; the frontend displays the results. If no events are available, an empty state should be shown. If the request fails, the application should display an error.

UC-02: Manage Event Information

Actor: Event Operator. The operator opens event management, creates or edits event information, submits the form, and receives a result. The backend validates the request and persists the change only when permitted and valid.

UC-03: Register for an Event or Manage a Ticket

Actor: Visitor. The visitor selects an event and initiates an available ticket operation. The system validates the request and stores the resulting ticket state if all business rules are satisfied. Payment and issuance details require source verification.

UC-04: Verify a Ticket and Record Attendance

Actor: Checking Staff. Staff identify a ticket, the system retrieves its information and checks applicable rules, then an authorized attendance operation stores the result. Invalid or previously processed tickets must be handled according to implemented rules.

UC-05: Manage Sponsorship Information

Actor: Sponsor or Authorized Operator. The user opens sponsorship functions, reviews or edits permitted information, submits changes, and receives confirmation. Invalid references should be rejected.

UC-06: Submit Feedback

Actor: Visitor. The visitor opens a feedback form, answers its questions, submits the form, and receives the result. Required-field validation and persistence behavior must follow the implementation.

UC-07: Participate in a Survey

Actor: Visitor or Eligible Participant. The participant opens a survey, answers questions, submits responses, and receives the result. Submission and analytics behavior require source verification.

2.3 Use Case Diagram

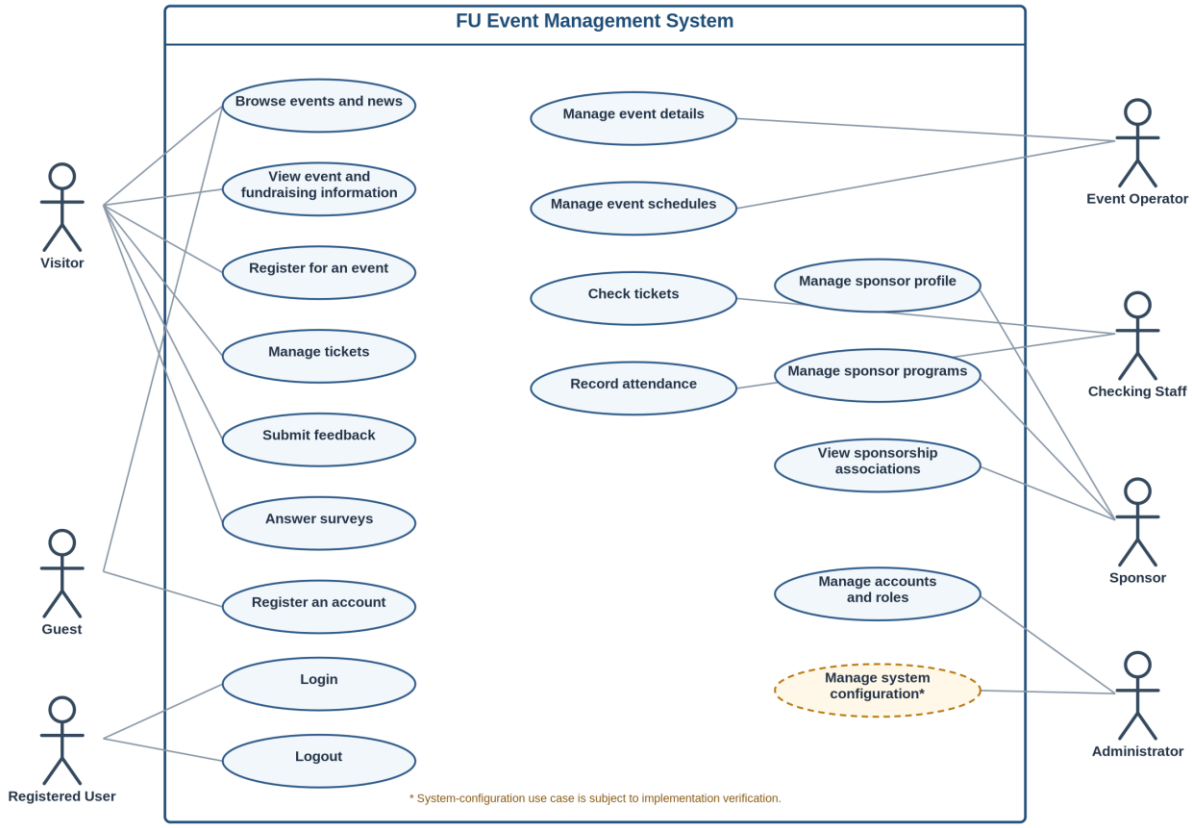


Figure 1. Use Case Diagram for FU Event Management.

The diagram reflects the baseline User Requirements Document. Guest registration and Visitor event registration are distinct use cases. The distinction between Guest, Registered User, and Visitor—and the Administrator system-configuration use case—should be checked against the running implementation.

3. Functional Requirements

3.1 System Functional Overview

3.1.1 Account and Access Management

- FR-01: The system shall allow Guests to view events and news.
- FR-02: The system shall provide an account-registration function for Guests.
- FR-03: The system shall support login and logout for Registered Users.
- FR-04: The system shall allow the Administrator to manage accounts and roles.
- FR-05: The system shall enforce role-based access to relevant functions according to the application's authorization rules.

3.1.2 Event Discovery and Management

- FR-06: Visitors shall be able to browse events and news.
- FR-07: Visitors shall be able to view event details.

- FR-08: Event information may include lifecycle/state, images, sale windows, start and end times, price, schedule entries, and fundraising information.
- FR-09: Event Operators shall be able to maintain event details and schedules.

3.1.3 Registration, Tickets, and Attendance

- FR-10: The system shall associate Visitors who register for events with ticket or registration records.
- FR-11: Ticket records shall track status and relevant timestamps.
- FR-12: Attendance records shall be associated with tickets and events and shall record attendance status.
- FR-13: Checking Staff may be assigned to events for attendance checking.

3.1.4 Sponsorship and Fundraising

- FR-14: Sponsor profiles shall be associated with accounts.
- FR-15: Sponsors may be associated with events and contribution or profit-sharing information.
- FR-16: Sponsor programs may be linked to one or more events.

3.1.5 Feedback and Surveys

- FR-17: Events may have feedback forms containing questions.
- FR-18: Visitor feedback answers shall be associated with a Visitor and a feedback question.
- FR-19: Events may have surveys containing questions and answer options or responses.

Implementation note: Detailed payment processing, donation transactions, exact ticket-state transitions, and survey analytics are not assumed by these requirements and must be verified against the source code.

NFR-01 to NFR-04 reflect the baseline User Requirements Document. NFR-05 to NFR-07 are additional report-level quality and verification criteria; confirm team acceptance before treating them as baseline requirements.

3.2 Non-Functional Requirements

ID	Requirement
NFR-01 — Security	Validate user input and enforce server-side authorization on protected operations.
NFR-02 — Data Integrity	Maintain referential integrity where foreign keys are defined.
NFR-03 — Configuration	Keep credentials and signing keys outside committed source files.
NFR-04 — Repeatability	Database setup shall be reproducible from a documented SQL script.
NFR-05 — Maintainability	Version-control code and documentation; update diagrams after schema changes.
NFR-06 — Usability	Forms should provide clear validation feedback and understandable operation results.
NFR-07 — Reliability	Failed requests must not be reported as successful; critical workflows should be tested.

3.3 Business Rules and Constraints

1. Account records must comply with the role relationship defined by the schema.
2. Ticket records must reference valid events and visitors where constraints are defined.
3. Attendance records must reference valid ticket records.
4. Event images and schedules must reference their corresponding events.
5. Sponsor-event and sponsor-program relationships must respect association-table constraints.
6. Feedback questions must reference feedback records; visitor answers must reference relevant questions and visitors.

7. Survey questions must reference surveys; survey answers must reference survey questions.
8. Ticket, event, and attendance state transitions must follow implemented business rules.
9. Authorization must be enforced by the backend, not only by frontend controls.

3.4 Requirements Traceability

Requirement Area	Database Area	Verification Method
Account/access	account, role	Entity and authorization review.
Events	event, state_event	API and persistence tests.
Event content	event_image, event_schedule	Relationship and CRUD tests.
Ticketing	visitor, ticket	Integration tests.
Attendance	attendance, ticket	Workflow and authorization tests.
Sponsorship	sponsor, sponsor_event, sponsor_program	Relationship tests.
Feedback	feedback, feedback_question, visitor_answer	Form and persistence tests.
Surveys	survey, survey_question, survey_answer	Survey workflow tests.

IV. SOFTWARE DESIGN DESCRIPTION

1. System Design

1.1 System Architecture

The application follows a logical three-layer architecture. This describes responsibilities, not a verified production deployment topology.

Layer	Technology	Responsibilities
Presentation	React, TypeScript, Vite, Tailwind CSS, Redux Toolkit	Render interfaces, collect input, manage client state, call APIs, show results and errors.
Application	Java, Spring Boot, RESTful APIs	Process HTTP requests, validate data, apply business rules, enforce access control, coordinate persistence.
Data	MySQL	Persist accounts, roles, events, visitors, tickets, attendance, sponsorship, feedback, and survey records.

Logical request flow: (1) user interacts with the frontend; (2) frontend sends an HTTP request; (3) backend validates the request and applies access rules; (4) backend reads or modifies MySQL records; (5) backend returns a response; (6) frontend displays the data or error.

1.2 Logical Architecture Diagram

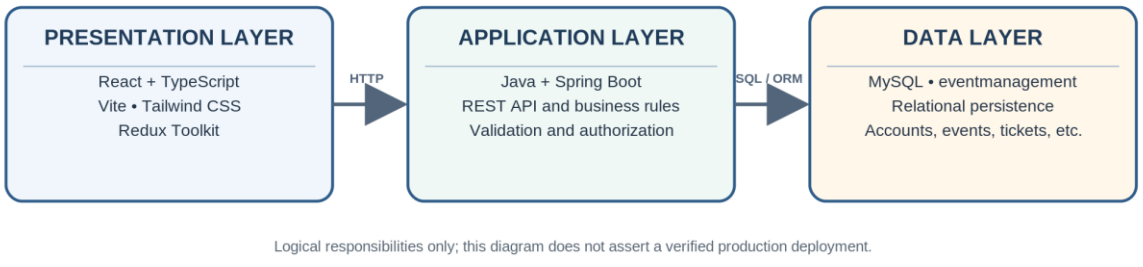


Figure 2. Logical Three-Layer System Architecture.

1.3 Technology Stack

Layer	Technology	Purpose
Frontend	React	User interface and component rendering.
Language	TypeScript	Frontend type checking.
Build tool	Vite	Development server and build tooling.
Styling	Tailwind CSS	Interface styling.
State management	Redux Toolkit	Client-side state management.
Backend	Java and Spring Boot	Server-side application and API infrastructure.
API	RESTful APIs	Frontend/backend communication.
Database	MySQL	Relational data storage.
Frontend hosting	Vercel	Frontend deployment platform as documented.

1.4 Authentication, Authorization, and Error Handling

The project description identifies authentication and role-based access control as intended capabilities. The backend should determine whether a request is authenticated and authorized. Frontend visibility rules are not a substitute for server-side authorization. The exact authentication mechanism, token lifecycle, role names, and permission assignments must be verified against the source code.

- Handle invalid input and missing records with clear responses.
- Reject unauthorized requests according to configured security rules.
- Handle database connectivity and persistence failures without false success messages.
- Avoid disclosing passwords, secrets, stack traces, or unnecessary internal details to users.

1.5 Security Considerations

- Keep database passwords and signing keys outside committed source files.
- Rotate credentials exposed in repositories, messages, or logs.
- Use secure password storage and server-side validation.
- Protect sensitive account and ticket information.
- Restrict cross-origin requests to appropriate origins in deployed environments.
- Use HTTPS in deployed environments and fictional data for local testing.

2. Database Design

2.1 Database Overview

The database is identified as eventmanagement and uses MySQL. It is intended to support identity management, event records, participant information, ticketing, attendance, sponsorship, feedback, and surveys. SQL definitions should be reviewed together with backend entity mappings and business logic.

2.2 Entity Classification

Category	Tables	Purpose
Identity and access	account, role	Accounts and role associations.
Events	event, state_event	Event information and lifecycle state.
Event content	event_image, event_schedule	Event images and schedule information.
User profiles	visitor, event_operator, checking_staff, sponsor	Role-related profile information.
Ticketing and attendance	ticket, attendance	Ticket and attendance records.
Financial summary	event_profit	Event-level financial summary.
Sponsorship	sponsor_event, sponsor_program, sponsor_program_event	Sponsor relationships and programs.
Feedback	state, feedback, feedback_question, visitor_answer	Feedback records, questions, and answers.

Category	Tables	Purpose
Surveys	survey, survey_question, survey_answer	Survey records, questions, and answers.

2.3 Overall Entity Relationship Diagram

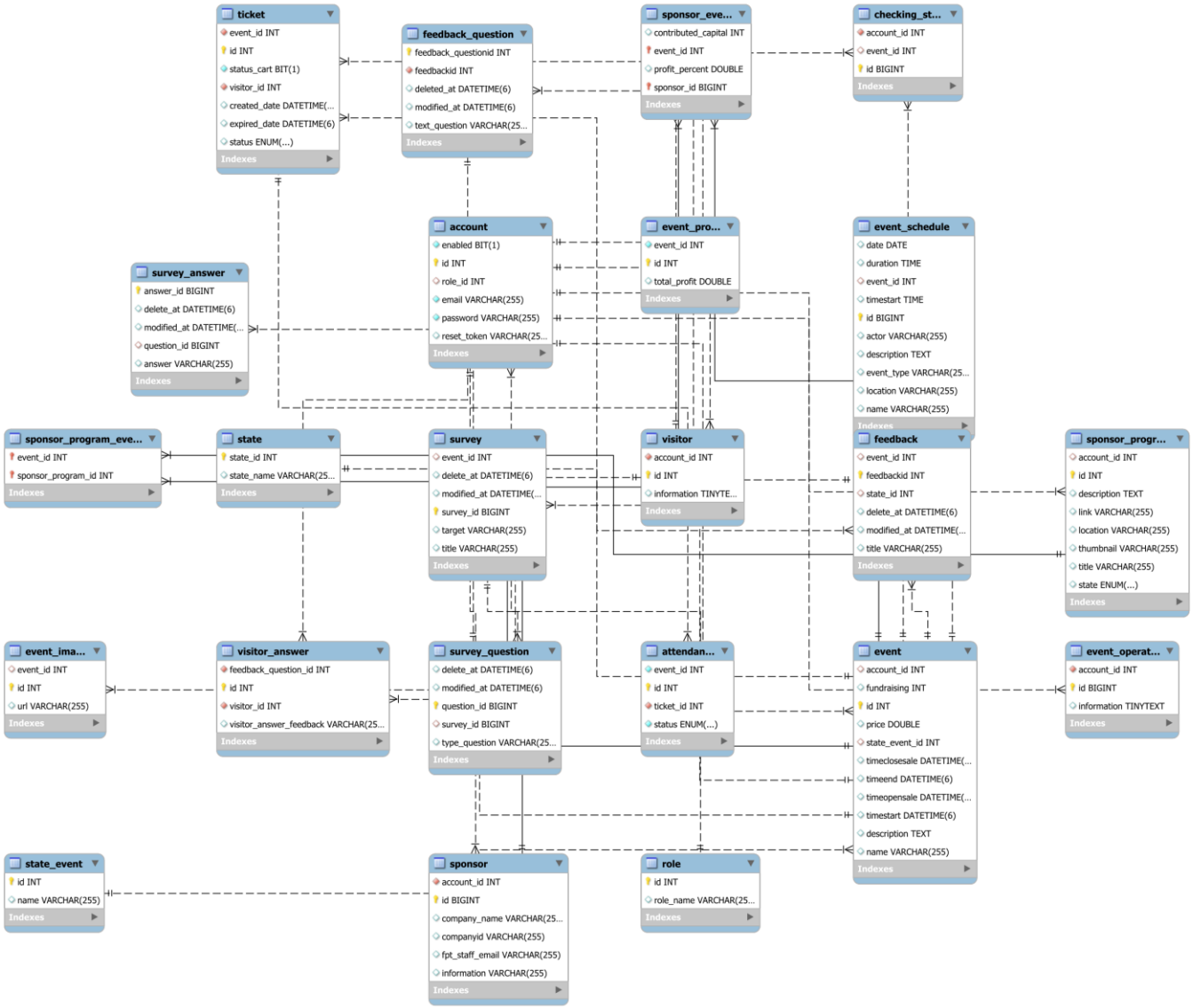


Figure 3. Overall Logical Database ERD (supplied schema model).

2.4 Main User Workflow Diagram

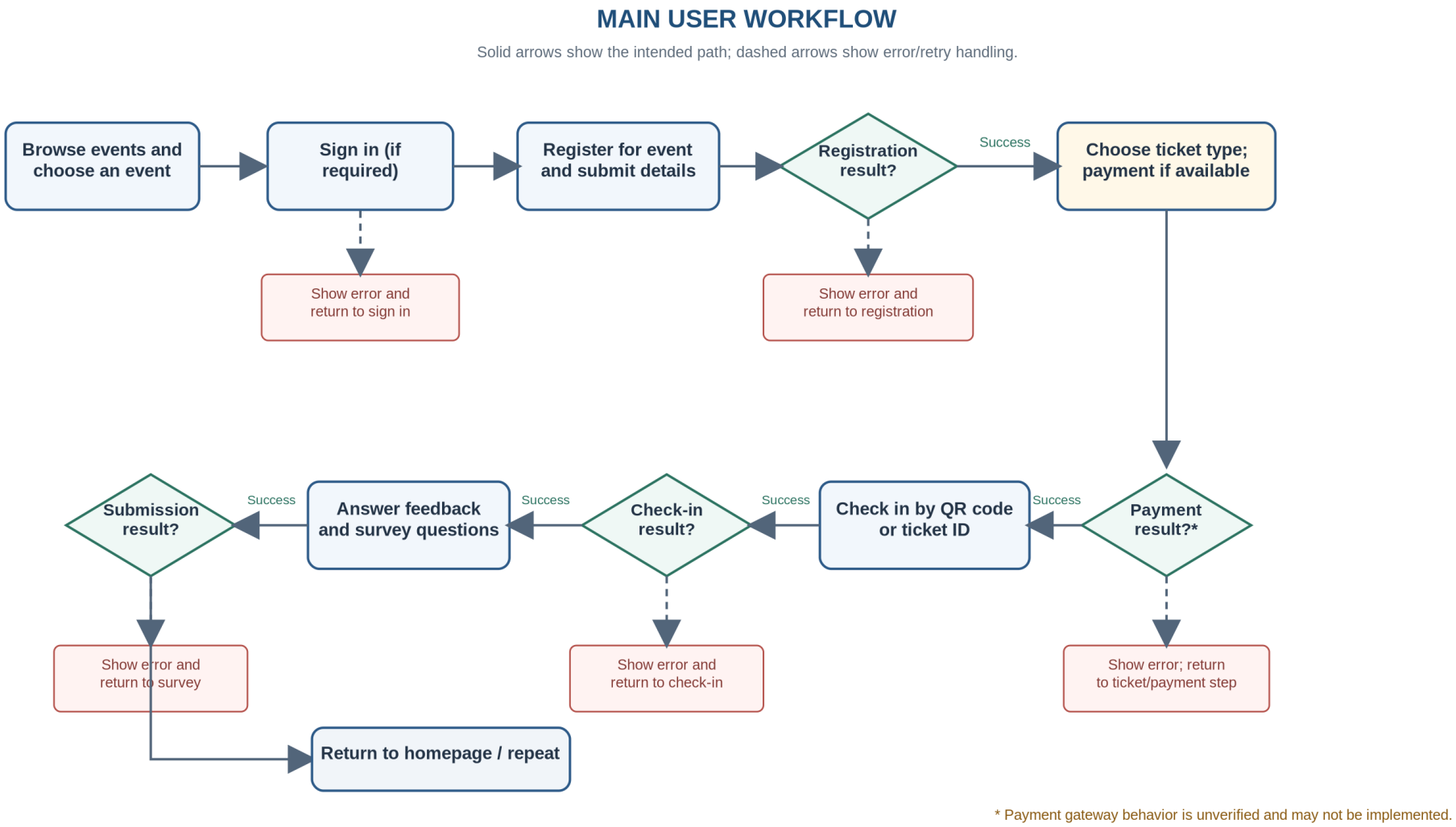


Figure 4. Main User Workflow Diagram (payment step subject to verification).

2.5 Principal Database Relationships

- account.role_id references role.id; event.account_id references account.id; event.state_event_id references state_event.id.
- Visitor, event_operator, checking_staff, and sponsor profile tables reference associated account records.
- event_image and event_schedule reference event; ticket references event and visitor; attendance references ticket.
- sponsor_event links event and sponsor using a composite key; sponsor_program_event links event and sponsor_program.
- feedback references event and state; feedback_question references feedback; visitor_answer references feedback_question and visitor.
- survey references event; survey_question references survey; survey_answer references survey_question.

Schema note: The supplied logical model displays relationship connectors, but the SQL foreign-key definitions remain authoritative. In particular, attendance.event_id is present in the schema, while the documented SQL does not define a foreign key from that column to event.id. Confirm whether that constraint should be added.

2.6 Data Dictionary

Table	Purpose
account	Stores account-related information.
role	Represents application roles.
event	Stores event information and related references.
state_event	Represents event status/lifecycle state.
event_image	Stores image records associated with events.
event_schedule	Stores event schedule information.
visitor	Stores visitor profile information.
event_operator	Stores event operator profile information.
checking_staff	Stores checking staff profile information.
sponsor	Stores sponsor profile information.
ticket	Stores ticket records associated with events and visitors.
attendance	Stores attendance information.
event_profit	Stores event-level financial summary information.
sponsor_event	Represents sponsor-event relationships.
sponsor_program	Stores sponsor program information.
sponsor_program_event	Associates sponsor programs with events.
state	Stores status information used by relevant schema relationships.
feedback	Stores feedback-related records.
feedback_question	Stores questions associated with feedback records.
visitor_answer	Stores visitor responses to feedback questions.
survey	Stores survey information.
survey_question	Stores questions associated with surveys.
survey_answer	Stores answer-related records associated with survey questions.

This is a high-level data dictionary. An implementation-level version should include exact column names, SQL types, nullability, defaults, keys, indexes, and constraints from the final SQL script.

2.7 Database Integrity and Validation

- Verify primary key and foreign key definitions and delete/update behavior.
- Check nullability, defaults, indexes, unique constraints, and ENUM values.
- Compare SQL schema definitions with JPA entity mappings.
- Review whether account email addresses require uniqueness.
- Confirm whether attendance.event_id should have an explicit foreign key.
- Clarify whether survey answer records represent selectable options, participant submissions, or both.
- Document how event_profit values are calculated and updated.
- Confirm ticket status transitions in application logic.

2.8 Database Installation Script

Recommended filename: database/create-database.sql. The script should include database selection/creation instructions, table definitions, primary and foreign key constraints, required indexes, defaults, optional fictional sample data, and comments for major sections.

Before execution, inspect all destructive statements. DROP TABLE IF EXISTS statements can delete existing table data. Run the script only against a disposable or backed-up local/development schema and record the result.

V. SOFTWARE TESTING DOCUMENTATION

1. Overall Description

1.1 Test Scope

Testing should cover database initialization, backend startup, frontend dependency installation, frontend/backend communication, event workflows, ticket and attendance operations, access control, feedback, and survey persistence.

1.2 Test Model

Use incremental verification: validate the database first, then backend startup and API behavior, then frontend integration, and finally representative end-to-end workflows. Test outcomes must be recorded only after execution.

1.3 Testing Levels

- Unit testing — individual methods, validators, and business rules.
- Integration testing — interactions between application components, repositories, and MySQL.
- API testing — HTTP requests, response codes, validation, and authorization.
- System testing — frontend, backend, and database working together.
- User acceptance testing — representative user workflows against intended requirements.

2. Test Plan

2.1 Test Cases

Test ID	Scenario	Expected Result
TC-DB-01	Execute SQL on a clean local schema.	Script completes without SQL errors.
TC-DB-02	Inspect created tables.	Expected tables and columns exist.
TC-DB-03	Insert a record with an invalid foreign key.	Database rejects it where a foreign key is defined.
TC-BE-01	Start backend with valid datasource settings.	Application starts and connects to MySQL.
TC-BE-02	Start backend with invalid datasource credentials.	Connection failure is reported.
TC-FE-01	Install frontend dependencies.	Installation completes without package parsing errors.
TC-FE-02	Start frontend development server.	Configured development server starts.
TC-EVT-01	Request available event records.	API returns a response consistent with its contract.
TC-EVT-02	Submit invalid event information.	Invalid input is rejected where validation applies.
TC-TKT-01	Request ticket for a valid event.	Operation succeeds only if business rules are satisfied.
TC-TKT-02	Request ticket for a nonexistent event.	Operation is rejected without an invalid reference.

Test ID	Scenario	Expected Result
TC-ATT-01	Record attendance for a valid ticket.	Operation succeeds if authorized.
TC-SEC-01	Attempt protected operation without authorization.	Request is rejected according to security rules.
TC-FBK-01	Submit feedback with invalid required fields.	Validation feedback is returned.
TC-SRV-01	Submit a valid survey response.	Behavior follows implemented survey rules.

2.2 Test Execution Record

Field	Description
Test ID	Unique test identifier.
Execution Date	Date the test was performed.
Environment	Java, MySQL, Node.js, and application versions.
Preconditions	Required accounts, data, and configuration.
Steps	Exact actions performed.
Expected Result	Intended behavior.
Actual Result	Observed behavior.
Status	Not Run, Pass, Fail, or Blocked.
Evidence	Logs, screenshots, or test output.
Defect Reference	Related issue, if any.

All proposed test cases should initially be marked Not Run until actual results have been collected.

2.3 Defect Management

Each defect should include a unique identifier, reproduction steps, expected behavior, actual behavior, severity, environment, and current status. Remove credentials from logs and screenshots. A defect should be considered resolved only after the reproduction steps have been repeated successfully.

VI. RELEASE PACKAGE AND USER GUIDES

1. Deliverable Package

- Frontend source repository: https://github.com/thungoc123/FE_FUEventManagement
- Backend source repository: https://github.com/thungoc123/BE_Event
- Project documentation repository: https://github.com/namyus/all-project-docs
- User Requirements Document: requirements/user-requirements.md
- Overall logical ERD and main user workflow diagram source files.
- Use Case Diagram source file.
- MySQL database creation script and schema documentation.
- Final report and test execution records.

2. Installation Guides

2.1 System Requirements

- Java Development Kit compatible with the backend build configuration.
- MySQL Server and a database client such as MySQL Workbench.
- Node.js and npm compatible with the frontend package configuration.
- Git and a compatible IDE/editor such as IntelliJ IDEA or Visual Studio Code.
- A modern web browser.

2.2 Setup Database

1. Start the local MySQL Server and connect using MySQL Workbench.

2. Back up any existing eventmanagement database that must be retained.
3. Open database/create-database.sql and inspect DROP TABLE IF EXISTS statements.
4. Execute the script against a local or development MySQL instance.
5. Refresh the schema tree and verify that expected tables exist.
6. Use fictional demo data only; placeholder passwords are not valid login credentials.

2.3 Setup Backend API

1. Open the backend repository in IntelliJ IDEA or another compatible Java IDE.
2. Review the build file to determine the required Java version and build tool.
3. Resolve project dependencies.
4. Configure datasource URL, username, and password using environment variables or an ignored local configuration file.
5. Confirm that SQL tables match the backend entity mappings.
6. For schema-first testing, set spring.jpa.hibernate.ddl-auto=none only after validating the schema.
7. Start Spring Boot and inspect logs for database connection or entity mapping errors.
8. Test an endpoint documented by the actual backend source code.

2.4 Setup Frontend Web Application

1. Open the frontend repository in a terminal and inspect package.json for scripts and dependencies.
2. Validate package.json syntax, then run npm install.
3. Set the frontend API base URL using the configuration already defined by the project.
4. Run the development script defined in package.json, commonly npm run dev.
5. Open the local URL printed by Vite.
6. Check browser network requests to confirm API calls target the intended backend.
7. Resolve API URL or CORS issues based on the actual configuration.

2.5 Environment Configuration

Local configuration may include server port, database URL and credentials, JWT signing material, SMTP credentials, and frontend API base URL. Do not place real credentials in public documentation, SQL scripts, reports, or screenshots. Rotate any credentials already exposed.

2.6 Troubleshooting

Problem	Possible Cause	Recommended Action
npm EJSONPARSE	Invalid JSON in package.json.	Correct missing commas, quotes, or braces.
MySQL connection failure	Incorrect URL/port or stopped service.	Verify MySQL Server and datasource settings.
Access denied	Incorrect credentials or insufficient privileges.	Check local account and database permissions.
Table does not exist	Script not executed or wrong schema selected.	Execute script and verify active schema.
Unknown column	SQL and entity mappings differ.	Compare SQL definitions with JPA mappings.
CORS error	Backend does not permit frontend origin.	Review existing CORS configuration.
HTTP 401 / 403	Authentication or authorization failure.	Review configured security rules and permissions.
Startup failure	Dependency, configuration, or mapping problem.	Review logs and build configuration.

3. User Manual

3.1 Overview

This guide describes intended user journeys. Exact page names, menu labels, screen layouts, and steps must be updated after reviewing the running frontend. These descriptions are not claims about specific UI controls.

3.2 Visitor Guide

3.2.1 Browse Events

1. Open the event listing page.
2. Review available event information.
3. Select an event to view details.
4. Review displayed schedule, price, and other available information.
5. Return to the listing when finished.

3.2.2 Manage a Ticket

1. Open an event that supports ticket-related operations.
2. Complete authentication if required.
3. Initiate the available registration or ticket operation.
4. Follow validation and confirmation messages.
5. Review the resulting ticket status.

3.2.3 Submit Feedback or Survey Responses

1. Open an available feedback form or survey.
2. Read the questions and enter the requested responses.
3. Review required fields.
4. Submit the form.
5. Confirm that the application reports the result.

3.3 Event Operator Guide

1. Authenticate with an account authorized for event management.
2. Open the event management function.
3. Select an existing event or initiate creation.
4. Enter the event information supported by the application.
5. Review required dates, prices, status, and other fields.
6. Submit changes and confirm the result.
7. Review the resulting event information.

3.4 Checking Staff Guide

1. Authenticate with an account permitted to perform attendance operations.
2. Open the ticket checking function.
3. Identify the relevant ticket and review its information.
4. Follow the supported attendance recording procedure.
5. Wait for confirmation.

6. If an error occurs, verify the ticket and operation status before retrying.

3.5 Sponsor and Administrator Guides

Sponsors should access only the sponsorship functions available to their account and update permitted information. Administrative activities may include account management, role assignment, and privileged event operations, but only permissions confirmed by the backend should appear in the final user guide.

VII. PROJECT EVALUATION, LIMITATIONS, AND FUTURE DEVELOPMENT

1. Project Evaluation

FU Event Management combines frontend development, backend API development, and relational database design. Its intended scope applies software engineering concepts including separation of frontend/backend responsibilities, RESTful communication, relational data modeling, account-role associations, ticketing, attendance, sponsorship, and structured feedback and survey records. Operational status must be evaluated against source code and test evidence.

2. Current Limitations

- The original historical report and complete historical documentation were not available as authoritative sources for this reconstruction.
- Some requirements are inferred from project scope and database structure.
- The exact API contract and security permission matrix require source-code verification.
- The SQL script must be executed on a clean local MySQL instance before being considered verified.
- Frontend/backend integration must be tested in the intended local environment.
- A public backend deployment is not documented in the supplied project materials.
- Payment, donation, financial reconciliation, and email workflows require additional verification.
- ERDs and the Use Case Diagram must be compared with the actual schema and implementation.
- Test results must be recorded only after execution.

3. Future Improvements

- Review database unique constraints, indexes, and foreign-key coverage.
- Standardize timestamp handling and document lifecycle/status rules.
- Add automated database integrity, unit, and integration tests.
- Review authentication, authorization, validation, and error handling.
- Document the API contract and separate secrets from source code.
- Improve frontend loading, empty, and error states and form validation.
- Maintain diagrams alongside SQL schema changes and add verified screenshots.
- Record test execution results and maintain a changelog for schema and API changes.

4. Conclusion

FU Event Management was designed as a web-based platform for organizing event information and supporting participant, ticket, attendance, sponsorship, feedback, and survey workflows. This report provides a structured reference for the intended scope, requirements, logical architecture, database design, testing approach, and local development setup. Further verification against the available source code, clean database installation, and documented test execution will improve accuracy and reproducibility.

REFERENCES

1. FU Event Management project README and documented technology stack.
2. FU Event Management frontend source repository: https://github.com/thungoc123/FE_FUEventManagement
3. FU Event Management backend source repository: https://github.com/thungoc123/BE_Event
4. Project documentation repository: https://github.com/namyus/all-project-docs
5. MySQL database creation script and schema documentation maintained with the project.
6. Spring Boot, React, TypeScript, Vite, and MySQL documentation relevant to the actual versions used.

APPENDIX A. DOCUMENTATION DELIVERABLE CHECKLIST

- Project README and User Requirements Document.
- Overall logical ERD — PNG and SVG.
- Main User Workflow Diagram — PNG and SVG.
- Use Case Diagram — PNG and SVG.
- MySQL database creation script and schema documentation.
- Backend and frontend installation guides.
- Verified test execution records.
- Final PDF report.

APPENDIX B. RECOMMENDED DOCUMENTATION STRUCTURE

```
FU-Event-Management/
├── README.md
├── requirements/
│   └── user-requirements.md
├── diagrams/
│   ├── overall-erd.mmd
│   ├── main-user-workflow.mmd
│   ├── use-case-diagram.puml
│   ├── overall-erd.png
│   ├── main-user-workflow.png
│   └── use-case-diagram.png
├── database/
│   ├── create-database.sql
│   └── schema-documentation.md
└── reports/
    ├── FU-Event-Management-Final-Report.docx
    └── FU-Event-Management-Final-Report.pdf
```